

StepCheck: Sound Static Verification of Deployed AWS Step Functions Workflows

Technical Report: Formal Development, Proofs, and Evaluation Detail

Nimród Földvári*

Florin Crăciun†

Faculty of Mathematics and Computer Science, Babes-Bolyai University, Cluj-Napoca, Romania

{harold.foldvari,mihai.craciun}@ubbcluj.ro

Abstract

This technical report is the companion to the paper *StepCheck: Sound Static Verification of Deployed AWS Step Functions Workflows* [1], which presents STEP CHECK, a static verifier for the Amazon States Language (ASL). The paper states its guarantees and gives their intuition; here we give the complete development behind them: the abstract interpretation underlying the sound data-flow provenance analysis—its domain of document-and-variable environments, concretization, transfer functions (including **Assign** and the modeled JSONata fragment), fixpoint, and the depth- k widening that guarantees convergence—the workflow type discipline for contracts and typestate, and the downstream-aware Saga-compensation check, with full proofs of all three central results. Theorems 1–3 below correspond, in order, to the main paper’s Theorem 1 (sound missing-field reports), the declared-tier consistency condition, and the relative soundness and completeness of SC4011; the paper states the last two in prose, and all three are stated here as theorems for a uniform formal treatment. We also give the SC4011 algorithm in full and note the conservative basis of the concurrency and temporal analyses. Beyond the formal development, this report carries the evaluation detail the paper cites but has no room for: the ASL-to-BPMN encoding and exact baseline configuration behind the comparison with workflow-soundness verifiers (Section 7), the mutation operators with per-class results for both the designed and the hard suites, and the calibration of the name-based inference (Section 8), and the sidecar syntax for declared premises (Section 9). Code references (e.g. SC1101) follow the main paper.

Contents

1	Scope and relation to the main paper	2
2	The data-flow abstract interpretation	3
2.1	The concrete property	3
2.2	Abstract domain	3
2.3	Concretization	4
2.4	Transfer functions	5
2.5	The fixpoint	8
2.6	Reference checking	9
2.7	Nested machines	11
2.8	The dead-guard refinement (SC1110)	11
2.9	Coverage of the modeled fragment	12
2.10	External validity: independent repositories	12

*ORCID 0009-0008-3531-0478

†ORCID 0000-0003-0335-8369

3	The workflow type discipline	13
3.1	Signatures, judgment, and validity	13
4	Downstream-aware Saga compensation	15
4.1	Model of failure and the post-commit region	15
4.2	The SC4011 algorithm	15
4.3	Reverse-order chains	16
5	Conservative analyses: concurrency and temporal (notes)	17
6	Artifact and implementation notes	18
7	Baselines: the Asl-to-BPMN encoding and tool configuration	19
7.1	The encoding	19
7.2	Crediting rule and tool configuration	20
8	Mutation operators, per-class results, and inference calibration	20
8.1	Operators	20
8.2	Per-class results	21
8.3	Calibration of the name-based inference	22
9	Sidecar syntax for declared premises	22
10	Extended limitations	23

1 Scope and relation to the main paper

The main paper keeps its result statements and one-paragraph intuitions inline and defers their proofs and the supporting formalism here, so the body stays within venue limits. This report adds nothing to the claims; it discharges them.

We assume the ASL data plane as fixed in the paper’s background [11]. A workflow is a state machine over eight state types (`Task`, `Choice`, `Parallel`, `Map`, `Wait`, `Pass`, `Succeed`, `Fail`). A single JSON document is threaded between states and reshaped by the five-stage I/O pipeline `InputPath` $\rightarrow$ `Parameters/ItemSelector` $\rightarrow$ `ResultSelector` $\rightarrow$ `ResultPath` $\rightarrow$ `OutputPath`. A `.$` field reference reads a `JSONPath` [9] into the current document, and the runtime *fails* the execution if such a reference resolves to an absent field. Alongside the document, an execution carries *workflow variables*: an `Assign` block binds names that later states read as `$x`, and a state may switch its `QueryLanguage` to `JSONata`, replacing the `.$` pipeline with `{% %}` expressions over the state input, the state result, the error output and those same variables. Both are in scope for the data-flow analysis of Section 2, which is therefore stated over document $\times$ variable environments rather than over a document alone.

The development below is faithful to the implementation (module `passes/dataflow.rs` for Section 2, `passes/contract.rs` and `passes/typestate.rs` for Section 3, and `passes/compensation.rs` for Section 4); the execution oracle (`concrete.rs`) that empirically witnesses Theorem 1 is described in the main paper. Nested machines (`Map` iterators and `Parallel` branches) are treated in Remark 2.

Table 1 records, for each fact a check needs, whether ASL exposes it natively and where `STEPCHECK` obtains it otherwise (summarized in prose in the main paper’s background section).

Fact a check needs	In Asl?	Source in StepCheck
Control flow and terminals	yes	native structural pass
JSONPath data movement	yes	native data-flow pass
Contracts and typestate	no	sidecar / DSL / schema metadata
Idempotency, persistence, compensator	no	declared premises; optional warning inference
Shared resources and time bounds	partial	conservative native passes

Table 1: Facts needed by STEPCheck and where they come from.

2 The data-flow abstract interpretation

The provenance analysis is a forward abstract interpretation [5] over a lattice of *environments*: a pair of JSON *shapes*, one abstracting the state document threaded through the workflow, one abstracting the store of **Assign**-bound workflow variables. It over-approximates, at each state, the set of environments with which the state may be entered, and flags a read site whose path resolves on none of them.

2.1 The concrete property

Fix a workflow with state set S . A concrete execution threads a JSON document and carries a *variable store*: a finite map from variable names to JSON values that starts empty and grows only through **Assign**. Write $\mathcal{D}$ for the set of JSON values and $obj(\nu)$ for the store ν viewed as the JSON object $\{x \mapsto \nu(x)\}$. For a state s , let $\llbracket s \rrbracket$ be the set of pairs (d, ν) with which s is entered over all executions that reach s . A path $p = k_1.k_2.\dots.k_m$ is *present* in a value d if d is an object with key k_1 whose value has k_2 , and so on to k_m ; otherwise p is *absent*.

A *read site* of s is a reference the runtime evaluates there and aborts on when it does not resolve, paired with the value it resolves against: the state document for a JSONPath document reference and for `$states.input`, the variable store for a `$`-variable reference, and the state’s own result or error output for `$states.result` and `$states.errorOutput`. The analysis answers a single question soundly: *is the read site’s path absent on every $(d, \nu) \in \llbracket s \rrbracket$?* If so, the reference fails on every execution reaching s , and STEPCheck reports SC1101.

2.2 Abstract domain

A *shape* abstracts a set of JSON values by the field paths that *may* be present. The same shapes abstract the variable store, read as an object whose top-level keys are variable names.

Definition 1 (Shapes). The set of shapes Σ is generated by

$$\sigma ::= \top \mid \text{Obj } r, \quad r : \text{Key} \rightarrow \Sigma \text{ a finite partial map.}$$

$\top$ is opaque (any field may be present). $\text{Obj } r$ constrains an object to the keys in $\text{dom}(r)$: every top-level key that *may* occur is listed, so a key outside $\text{dom}(r)$ is present on no represented *object*. The abstraction functions send a scalar to $\text{Obj } \emptyset$ (no fields) and an array to $\top$ (its elements are unconstrained); this is a choice of the abstraction, not a claim that $\gamma(\text{Obj } r)$ contains only objects—indeed $\gamma(\text{Obj } r)$ admits every non-object (Definition 5), so $\text{Obj } r$ is a constraint on the object case alone.

Definition 2 (Order and join). The join $\sqcup$ is

$$\top \sqcup \sigma = \sigma \sqcup \top = \top, \quad \text{Obj } r_1 \sqcup \text{Obj } r_2 = \text{Obj } r, \quad \text{dom}(r) = \text{dom}(r_1) \cup \text{dom}(r_2),$$

where $r(k) = r_1(k) \sqcup r_2(k)$ if $k \in \text{dom}(r_1) \cap \text{dom}(r_2)$, and $r(k) = r_i(k)$ if k occurs only in r_i . The induced order is $\sigma_1 \sqsubseteq \sigma_2 \iff \sigma_1 \sqcup \sigma_2 = \sigma_2$; equivalently $\sigma \sqsubseteq \top$ for all σ , and $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ iff $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$ for all $k \in \text{dom}(r_1)$.

Intuitively a shape is *higher* when it admits *more* documents: $\top$ is the top, $\text{Obj } \emptyset$ the most precise object shape, and the join is the least upper bound. The join is exactly the key-union of the implementation (clone the left record, then for each key of the right record join or insert).

Definition 3 (Environments). An *environment* is a pair $\varepsilon = \langle \sigma_d, \sigma_v \rangle$ in $\mathcal{E} = \Sigma \times \Sigma$, where σ_d abstracts the state document and σ_v abstracts the variable store viewed as an object. A top-level key of σ_v is therefore a variable name, and the variable reference $\$x.a.b$ is the path $x.a.b$ into σ_v : one lattice serves both components, and only the *reference syntax* decides which of them a read site addresses. Order, join and widening are componentwise,

$$\langle \sigma_d, \sigma_v \rangle \sqsubseteq \langle \sigma'_d, \sigma'_v \rangle \iff \sigma_d \sqsubseteq \sigma'_d \wedge \sigma_v \sqsubseteq \sigma'_v, \quad \langle \sigma_d, \sigma_v \rangle \sqcup \langle \sigma'_d, \sigma'_v \rangle = \langle \sigma_d \sqcup \sigma'_d, \sigma_v \sqcup \sigma'_v \rangle,$$

so $\mathcal{E}$ is again a lattice, and every lemma below about a single shape applies to either component.

Definition 4 (Definite absence). A path $p = k_1 \dots k_m$ is *definitely absent* in σ if descending σ by $k_1, \dots$ reaches some $\text{Obj } r$ with $k_i \notin \text{dom}(r)$ *before* encountering $\top$. Descent stops at $\top$: a path under $\top$ is never definitely absent.

2.3 Concretization

Definition 5 (Concretization γ). $\gamma : \Sigma \rightarrow \mathcal{P}(\mathcal{D})$ is

$$\gamma(\top) = \mathcal{D}, \quad \gamma(\text{Obj } r) = \{ d \in \mathcal{D} \mid d \text{ object} \Rightarrow \forall k \in \text{keys}(d). k \in \text{dom}(r) \wedge d(k) \in \gamma(r(k)) \}.$$

A non-object d (scalar, array) has no top-level keys, so it lies in $\gamma(\text{Obj } r)$ for every r .

Definition 6 (Environment concretization Γ). Writing *Store* for the set of variable stores, $\Gamma : \mathcal{E} \rightarrow \mathcal{P}(\mathcal{D} \times \text{Store})$ is

$$\Gamma(\langle \sigma_d, \sigma_v \rangle) = \{ (d, \nu) \mid d \in \gamma(\sigma_d) \wedge \text{obj}(\nu) \in \gamma(\sigma_v) \}.$$

A variable store is always an object, so on the second component the object premise of γ is never vacuous: a variable name outside $\text{dom}(\sigma_v)$ is bound in *no* store that Γ admits. The variable component is thus exact at its top level, where the document component must also admit non-objects.

Lemma 1 (Absence is captured). *If p is definitely absent in σ , then p is absent in every $d \in \gamma(\sigma)$.*

Proof. Induction on $|p|$. If $p = k_1 \dots$ and $\sigma = \text{Obj } r$ with $k_1 \notin \text{dom}(r)$, then by definition of γ no $d \in \gamma(\text{Obj } r)$ has top-level key k_1 , so p is absent in d . Otherwise $k_1 \in \text{dom}(r)$ and $k_2 \dots k_m$ is definitely absent in $r(k_1)$; for any $d \in \gamma(\text{Obj } r)$, either d lacks k_1 (so p absent) or $d(k_1) \in \gamma(r(k_1))$, and by the induction hypothesis $k_2 \dots k_m$ is absent in $d(k_1)$, so p is absent in d . Definite absence cannot pass through $\top$, so the case $\sigma = \top$ never arises along the descent. $\square$

Lemma 2 (γ is monotone, $\sqcup$ is sound). $\sigma_1 \sqsubseteq \sigma_2 \Rightarrow \gamma(\sigma_1) \subseteq \gamma(\sigma_2)$, and $\gamma(\sigma_1) \cup \gamma(\sigma_2) \subseteq \gamma(\sigma_1 \sqcup \sigma_2)$.

Proof. Monotonicity is by induction on shape structure: if $\sigma_2 = \top$ it is immediate; for $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ we have $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$, so any d admitted by $\text{Obj } r_1$ has $\text{keys}(d) \subseteq \text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $d(k) \in \gamma(r_1(k)) \subseteq \gamma(r_2(k))$ by the induction hypothesis. Soundness of $\sqcup$ follows since $\sigma_i \sqsubseteq \sigma_1 \sqcup \sigma_2$ and γ is monotone. $\square$

Because $\sqsubseteq$ and $\sqcup$ on $\mathcal{E}$ are componentwise, both lemmas lift to environments and Γ at once: $\varepsilon_1 \sqsubseteq \varepsilon_2 \Rightarrow \Gamma(\varepsilon_1) \subseteq \Gamma(\varepsilon_2)$, and $\Gamma(\varepsilon_1) \cup \Gamma(\varepsilon_2) \subseteq \Gamma(\varepsilon_1 \sqcup \varepsilon_2)$.

2.4 Transfer functions

Each stage is a monotone function on shapes; a state’s transfer is a monotone function on environments. We write $\text{narrows}(\sigma, \pi)$ for re-rooting along a path expression π (used by `InputPath/OutputPath`), $\text{ctor}(v)$ for the record built from a constructor value v (used by `Parameters/ItemSelector/ResultSelector`), $\text{place}(\sigma, \rho, \sigma_r)$ for merging a result shape σ_r at a `ResultPath` ρ , $\text{res}(r, \varepsilon)$ for resolving a reference against an environment, and asg for the effect of `Assign` on the variable component.

Re-rooting. Write ϵ for the empty (absent) path and $k :: \text{rest}$ for the split of a path into its first key and remainder. Then $\text{narrows}(\sigma, \epsilon) = \sigma$ (no path) and $\text{narrows}(\sigma, \$) = \sigma$ (the bare root); $\text{narrows}(\sigma, \text{null}) = \text{Obj } \emptyset$, sound because ASL maps an `InputPath/OutputPath` of `null` to the empty object $\{\} \in \gamma(\text{Obj } \emptyset)$. For a simple path $\pi = k_1 \dots k_n$, narrows descends via nav : $\text{nav}(\top, _) = \top$ and $\text{nav}(\text{Obj } r, k :: \text{rest}) = \text{nav}(r(k), \text{rest})$ if $k \in \text{dom}(r)$, and $= \top$ otherwise (a missing intermediate key yields $\top$, over-approximating). Any path outside the simple-path fragment (wildcards, filters, functions, brackets, the `$$` context) yields $\top$; a non-string, non-`null` path value is treated as the identity ($\text{narrows}(\sigma, \cdot) = \sigma$), a case ASL deploy-time validation rejects and which is therefore unreachable in practice.

Constructors and literals. Two builders recurse over a JSON value v . The *constructor* $\text{ctor}(v)$ models a payload template (`Parameters/ItemSelector/ResultSelector`): for an object it builds `Obj` r where a key “ $k.\$$ ” contributes $r(k) = \top$ (it *reads* an opaque value into field k), a static key k contributes $r(k) = \text{ctor}(v(k))$, a scalar leaf is `Obj` $\emptyset$, and an array is $\top$. The *literal* builder $\text{lit}(v)$ models verbatim data (a `Pass Result`), which ASL copies uninterpreted: it agrees with ctor *except* that it does no `.$` stripping—a key literally named “ $k.\$$ ” stays a key of that exact name. Distinguishing the two is necessary for soundness: for `Result` = $\{\text{“a.\$”}: v\}$ the concrete document has the top-level key “ $\text{a.\$}$ ”, which is *not* in $\text{dom}(\text{ctor}) = \{a\}$, so ctor would under-approximate the key set.

Reference resolution. A reference string r at a read site denotes a path into one component of the environment $\varepsilon = \langle \sigma_d, \sigma_v \rangle$:

$$\text{res}(r, \varepsilon) = \begin{cases} \text{nav}(\sigma_d, \pi) & r = \$ \text{ or } r = \$.\pi, \pi \text{ simple,} \\ \text{nav}(\sigma_v, x.\pi) & r = \$x \text{ or } r = \$x.\pi, x \text{ a variable name,} \\ \top & \text{otherwise.} \end{cases}$$

The middle case is ASL’s variable syntax: a reference beginning with `$` that is neither the bare root `$`, nor a document path `$. . .`, nor the context object `$$`, has a variable name as its first segment. Classifying it explicitly—rather than letting it fall out of the `JSONPath` parser as an unrecognised string—is what turns a variable read from an untracked case into a checked one. The $\top$ case collects the context object, `States.*` intrinsics, non-`$` literals, and any path outside the simple-path fragment.

Result and merge. The result shape σ_{res} of a state, built from its effective input eff , is: $\top$ for an opaque `Task/Map/Parallel` result; $\text{ctor}(\text{ResultSelector})$ when a `ResultSelector` pins it; and for a `Pass`, $\text{lit}(\text{Result})$ if it carries a literal `Result`, else $\text{ctor}(\text{Parameters})$ if it carries only `Parameters`, else eff itself (a bare `Pass` forwards its effective input unchanged). Merging is

$$\text{place}(\sigma, \rho, \sigma_r) = \begin{cases} \sigma_r & \rho = \text{Default or } \rho = \$ \text{ (result replaces the document),} \\ \sigma & \rho = \text{Discard (null) (result dropped, input kept),} \\ \text{place}_{\text{in}}(\sigma, \pi, \sigma_r) & \rho = \text{Path}(\pi), \end{cases}$$

where $place_{in}$ descends π , cloning each record and *preserving siblings*, and writes σ_r at the leaf; a $\top$ parent stays $\top$, and an unparseable π yields $\top$. (This is why a field can survive a task that never mentions it: a sub-path merge leaves the rest of the document intact.)

Variable binding (Assign). An **Assign** block A is a finite map from keys to values: a key “ $n.\$$ ” binds variable n to what a JSONPath reference selects, a plain key n binds it to a constructed value. Its transfer on the variable component is

$$asg(\sigma_v, A, \sigma_s) = \sigma_v[n_i \mapsto \tau_i]_{i=1..|A|}, \quad \tau_i = \begin{cases} res(r_i, \langle \sigma_s, \sigma_v \rangle) & \text{key } n_i.\$, \text{ string value } r_i, \\ ctor_{asg}(v_i, \sigma_s, \sigma_v) & \text{key } n_i, \text{ value } v_i, \end{cases}$$

where $\sigma_v[n \mapsto \tau] = \top$ when $\sigma_v = \top$ and otherwise the record σ_v with n remapped to τ , and $ctor_{asg}$ is the constructor builder *except* that a nested “ $k.\$$ ” key contributes $res(r, \langle \sigma_s, \sigma_v \rangle)$ rather than $\top$: **Assign** propagates the shape of what it reads, which is what lets a later read of that variable be resolved precisely instead of hitting an opaque binding. Three points carry the soundness.

- *Simultaneity.* Every right-hand side is evaluated in the *pre-state* σ_v , not in the partially updated one: ASL evaluates a whole **Assign** block against the state as it stands before any of its bindings take effect, so the transfer is simultaneous rather than sequential.
- *Strong update.* The binding *replaces* n ’s shape rather than joining with it. This is sound because the block executes on every execution that leaves the state along this edge; the merging of alternative paths happens at the successor, in the fixpoint, where $\sqcup$ unions the key sets. A variable bound on only one of two incoming paths is therefore *may-present* at the merge, and definite absence survives only for a name that *no* path binds.
- *The source document.* The document that $\$$ denotes inside **Assign** is state-dependent, because ASL applies **Assign** at different points of the pipeline: for a **Task** it is the raw task result (a declared output schema or a known service envelope, else $\top$), for **Map** and **Parallel** it is $\top$, for a **Pass** the **Pass** result, and for **Choice** and **Wait** the effective (post-**InputPath**) input. Inside a **Catch** rule’s **Assign** it is the error output, hence $\top$.

JSONata states. A state may set **QueryLanguage:** **JSONata**, replacing the $.\$$ pipeline with $\{ \% \% \}$ expressions over four sources: $\$states.input$ (the state document), $\$states.result$ (the state’s own result), $\$states.errorOutput$, and the workflow variables $\$x$. Write

$$\mathcal{S}(\varepsilon, s) = \{ \text{input} \mapsto \sigma_d, \text{result} \mapsto \sigma_{res}, \text{errorOutput} \mapsto \top, \text{vars} \mapsto \sigma_v \}$$

for their shapes at s , with σ_{res} the task-result shape ($\top$ unless a declared output schema or a known service envelope pins it, and $\top$ for **Map** and **Parallel**) and the error output always $\top$. The expression abstraction is

$$\mathcal{J}[e]_s = \begin{cases} nav(\mathcal{S}(c), \pi) & e \text{ a bare reference to source } c \text{ at simple path } \pi, \\ \top & e = \$states.context \dots, \\ \text{Obj} \{ k_j \mapsto \mathcal{J}[e_j]_s \} & e = \{ 'k_1':e_1, \dots, 'k_n':e_n \}, \\ \top & \text{otherwise,} \end{cases}$$

and it lifts to a JSON value by $\mathcal{V}[\cdot]_s$: a string “ $\{ \% e \% \}$ ” takes $\mathcal{J}[e]_s$, any other scalar is the leaf $\text{Obj} \emptyset$, an object recurses keywise, and an array is $\top$. Note what the fragment covers and what it does not: a bare reference and an object constructor are modeled exactly, and *every*

other expression—arithmetic, function calls, predicates, string operations—is $\top$ and can produce no finding.

A JSONata state's output document is $\mathcal{V}[\![\text{Output}]\!]_{\mathcal{S}}$ when it declares an **Output**, and otherwise σ_{res} for a **Task**, $\top$ for **Map** and **Parallel**, and the entry document σ_d for the remaining kinds, which contribute no result of their own. Its **Assign** binds each variable to $\mathcal{V}[\![\cdot]\!]_{\mathcal{S}}$ of the assigned value, by the same strong update as above.

State transfer and error edges. For a state s with entry environment $\varepsilon = \langle \sigma_d, \sigma_v \rangle$, the output environment is $out(s, \varepsilon) = \langle \sigma'_d, \sigma'_v \rangle$. Its variable component is the **Assign** transfer, $\sigma'_v = asg(\sigma_v, \text{Assign}, \sigma_s)$, and its document component is

$$\sigma'_d = \begin{cases} \text{the JSONata output above} & s \text{ in JSONata mode,} \\ narrow(place(\sigma_d, \rho, \sigma_{res}), \text{OutputPath}) & \text{Task/Pass/Map/Parallel,} \\ narrow(eff, \text{OutputPath}) & \text{otherwise,} \end{cases}$$

with $eff = narrow(\sigma_d, \text{InputPath})$ the **InputPath**-selected input from which the result shape σ_{res} is built, ρ the state's **ResultPath**, and σ_s the **Assign** source document of the previous paragraph. The result is merged into the *raw* entry shape σ_d , *not* eff —exactly as ASL writes **ResultPath** into the pre-**InputPath** document, so a field the **InputPath** selection dropped still survives. A **Catch** rule c leaving a state with entry ε propagates

$$catch(s, \varepsilon, c) = \langle place(\sigma_d, \rho_c, \top), asg(\sigma_v, \text{Assign}_c, \top) \rangle :$$

the error object is opaque ($\top$), merged at the catcher's **ResultPath**, and the catcher's own **Assign**, if it has one, binds against that opaque error output. The variable store is otherwise carried across the error edge unchanged, since a failed state's own bindings do not take effect. A **Retry** re-executes the same state on the same entry environment, so it leaves entry shapes unchanged and the analysis ignores it.

Lemma 3 (Local soundness). *Writing a subscript c for the concrete counterpart of each stage, every transfer is sound on its own arguments:*

(re-root) $d \in \gamma(\sigma) \Rightarrow narrow_c(d, \pi) \in \gamma(narrow(\sigma, \pi));$

(build) *the concrete payload built over a document d from a template v lies in $\gamma(ctor(v))$, and from a literal v in $\gamma(lit(v))$;*

(merge) $d \in \gamma(\sigma) \wedge d_r \in \gamma(\sigma_r) \Rightarrow place_c(d, \rho, d_r) \in \gamma(place(\sigma, \rho, \sigma_r));$

(resolve) $(d, \nu) \in \Gamma(\varepsilon) \Rightarrow$ *the value a reference r selects in (d, ν) , where it resolves, lies in $\gamma(res(r, \varepsilon))$;*

(assign) $(d, \nu) \in \Gamma(\varepsilon) \wedge d_s \in \gamma(\sigma_s) \Rightarrow obj(asg_c(\nu, A, d_s)) \in \gamma(asg(\sigma_v, A, \sigma_s));$

(jsonata) *if every source of $\mathcal{S}$ over-approximates its concrete counterpart, then the value of a JSONata expression e , where defined, lies in $\gamma(\mathcal{J}[\![e]\!]_{\mathcal{S}})$, and likewise for $\mathcal{V}[\![\cdot]\!]_{\mathcal{S}}$ on a JSON value;*

(state/catch) $(d, \nu) \in \Gamma(\varepsilon) \Rightarrow out_c(s, (d, \nu)) \in \Gamma(out(s, \varepsilon))$ and $catch_c(s, (d, \nu), c) \in \Gamma(catch(s, \varepsilon, c)).$

Proof. By cases. *Re-rooting:* if $\sigma = \top$ or π is non-simple, $narrow(\sigma, \pi) = \top$ and $\gamma(\top) = \mathcal{D}$; for a simple path, nav keeps $d(k) \in \gamma(r(k))$ by Definition 5, and a missing intermediate key yields $\top$ (vacuously sound); $narrow(\sigma, \text{null}) = \text{Obj } \emptyset$ is sound because the selected concrete value is the empty object $\{\}$, which has no keys and hence lies in $\gamma(\text{Obj } \emptyset)$. *Constructor:* the concrete payload object has top-level keys exactly the template's keys $\subseteq \text{dom}(ctor(v))$; a $.\$$ key binds a runtime-read value in $\gamma(\top)$, and a static key's value lies in $\gamma(ctor(v(k)))$ by induction. *Literal:* the concrete value is v copied verbatim, whose keys are exactly $\text{dom}(lit(v))$ —including any literal

“ $\$.$ ” key—and each child lies in the corresponding γ by induction. *Result*: an opaque result lies in $\gamma(\top)$; a pinned **ResultSelector** reduces to the constructor case; a bare **Pass** forwards eff , and $d \in \gamma(eff)$ by hypothesis. *Merge*: $\rho \in \{\text{Default}, \$\}$ replaces, and the concrete output is the result, which lies in $\gamma(\sigma_r)$; $\rho = \text{Discard}$ keeps the input; for $\rho = \text{Path}(\pi)$, $place_{in}$ writes σ_r at π and preserves siblings, matching the concrete merge, and a $\top$ parent (or unparseable π) stays $\top$ (sound).

Resolve: a document reference descends d by nav and is sound by the re-rooting case, since $d \in \gamma(\sigma_d)$. A variable reference $\$x.\pi$ descends $obj(\nu)$ along $x.\pi$, and $obj(\nu) \in \gamma(\sigma_v)$ by Definition 6, so the identical argument applies to the second component. Every other reference yields $\top$.

Assign: the concrete block binds each n_i to the value of its right-hand side evaluated in the pre-state, which lies in $\gamma(\tau_i)$ —by (resolve) for a “ $n_i.\$$ ” key, and by (build) together with (resolve) at its $.\$$ leaves for a constructed value. The resulting store object has top-level keys $keys(obj(\nu)) \cup \{n_1, \dots\} \subseteq \text{dom}(\sigma_v) \cup \{n_1, \dots\}$, which is exactly $\text{dom}(asg(\sigma_v, A, \sigma_s))$; a name the block does not bind keeps both its old value and its old shape, and a name it binds gets τ_i on both sides. If $\sigma_v = \top$ the result is $\top$ and the claim is vacuous.

JSONata: structural induction on e . A bare reference navigates a source that over-approximates its concrete counterpart by hypothesis, so the re-rooting argument applies; **\\$states.context** and every unmodeled expression yield $\top$. An object constructor evaluates to an object whose keys are a *subset* of $\{k_1, \dots, k_n\}$ —JSONata drops a key whose value is **undefined**—with each present value in $\gamma(\mathcal{J}[e_j]_S)$ by the induction hypothesis; that subset inclusion is precisely what γ ’s “may occur” reading asks for, so the constructor shape is sound even though it lists keys the concrete object need not carry. For $\mathcal{V}[\cdot]_S$: a literal scalar has no top-level keys and so lies in $\gamma(\text{Obj } \emptyset)$, an object recurses by the induction hypothesis, and an array yields $\top$.

State transfer composes the above on the document component—with the JSONata output covered by (jsonata)—and applies (assign) on the variable component; the two components are independent, so Γ -membership follows from the two γ -memberships. *Catch* places $\top$ (the error object) at ρ_c , sound since $\gamma(\top) = \mathcal{D}$, and applies (assign) with $\sigma_s = \top$. $\square$

2.5 The fixpoint

The analysis maintains a *partial* map $\Phi : S \rightarrow \mathcal{E}$, seeding only the start state with $\nabla_k(\varepsilon_0)$; a state acquires an environment the first time an edge reaches it, and a state never reached stays outside $\text{dom}(\Phi)$ and is read as $\langle \top, \top \rangle$ at check time (an unreached state constrains nothing, and reporting against $\top$ never fires). To keep the domain finite-height on every topology the iteration *widens*: the depth- k operator ∇_k replaces every sub-record nested deeper than k by $\top$ — $\nabla_k(\top) = \top$, $\nabla_0(\text{Obj } r) = \top$, and $\nabla_k(\text{Obj } r) = \text{Obj } r'$ with $\text{dom}(r') = \text{dom}(r)$ and $r'(a) = \nabla_{k-1}(r(a))$ for $k > 0$ —applied componentwise to an environment, and the implementation fixes $k = 12$. Widening is monotone and *extensive* ($\sigma \sqsubseteq \nabla_k(\sigma)$), and its image (shapes of depth at most k) is closed under $\sqcup$. One *round* updates, for every $s \in \text{dom}(\Phi)$ with current entry $\Phi(s)$: for each normal successor t , $\Phi(t) \leftarrow \nabla_k(\Phi(t) \sqcup \nabla_k(\text{out}(s, \Phi(s))))$ (inserting the widened contribution when $t \notin \text{dom}(\Phi)$); for each **Catch** rule c with target u , $\Phi(u) \leftarrow \nabla_k(\Phi(u) \sqcup \nabla_k(\text{catch}(s, \Phi(s), c)))$. Updates are in place (Gauss–Seidel) in an unspecified state order; the fixpoint, once reached, is independent of that order. Rounds repeat until a round changes nothing.

Lemma 4 (Convergence and soundness of widening). *The forward iteration is monotone—each $\Phi(s)$ only ascends under $\sqsubseteq$ —and reaches a fixpoint on every workflow topology, cyclic ones included. That fixpoint is a post-fixpoint of the un-widened transfer functions, so it over-approximates every concrete environment reaching each state; widening costs precision but never soundness.*

Proof. Every update is a join followed by ∇_k , both monotone (Lemma 2) and both componentwise, so each component of each $\Phi(s)$ is non-decreasing. Let κ be the size of the workflow’s *key universe*: the finite set of keys occurring in any constructor (**Parameters**, **Result**, **ResultSelector**, **ItemSelector**, **Arguments**, **Output**, **Items**), in any **Assign** block—including the variable names it binds—in any JSONata object constructor, in the declared input schema, or as a segment of a **ResultPath** or catch **ResultPath**. Every transfer draws its keys from κ —a missing key, a non-simple path or an unmodeled expression yields $\top$, which adds none, and the variable component’s top-level keys are exactly the names some **Assign** binds—and ∇_k caps nesting at depth k , so both components of every reachable environment lie in the *finite-height* lattice of records of depth at most k over the κ -key universe, and $\mathcal{E}$, a product of two such lattices, has finite height as well. A non-decreasing chain in a finite-height lattice stabilises, so the iteration reaches a fixpoint in a bounded number of rounds; the implementation caps the loop at $N = |S| \cdot (\kappa_0 + 2) \cdot (k + 1) + 2$ rounds (with κ_0 its key count and $|S|$ the state count) purely as a defensive assertion that, by the finite-height argument, is never reached (Remark 1).

For soundness, extensivity gives $\varepsilon \sqsubseteq \nabla_k(\varepsilon)$, so a fixpoint Φ of the widened operator satisfies $\text{out}(s, \Phi(s)) \sqsubseteq \nabla_k(\text{out}(s, \Phi(s))) \sqsubseteq \Phi(t)$ for every normal edge $s \rightarrow t$, and likewise $\text{catch}(s, \Phi(s), c) \sqsubseteq \Phi(u)$ for every catch edge: Φ is a *post-fixpoint* of the un-widened transfers. The soundness induction of Theorem 1 uses only that each round joins a sound contribution into $\Phi(t)$, which this post-fixpoint property preserves, so Φ over-approximates $\llbracket s \rrbracket$ at every state. Finally ∇_k only enlarges shapes—it turns some **Objr** into $\top$ —and a path under $\top$ is never definitely absent, so widening can suppress a finding but never manufacture a false one. $\square$

Remark 1 (The widened bound is never binding in practice). Depth- k widening restores a finite-height domain on *every* topology, so convergence needs no restriction to acyclic or otherwise “bounded-depth” workflows: cyclic re-embedding—a bare **Pass** that forwards its narrowed input while a **ResultPath** re-embeds it one level deeper each iteration—is collapsed to $\top$ once it passes depth k . Empirically the bound N is far from tight: across the 193 AWS and 66 CNCF workflows (786 fixpoints, native and typed seeding, nested machines included) the iteration takes at most 7 rounds, and the acyclic scale sweep converges in a single forward pass by construction, so the defensive cap is never hit and no machine is ever suppressed. The depth $k = 12$ comfortably exceeds the deepest constructor nesting in the corpora, so on them widening is inert and loses no precision; it earns its keep only on the cyclic-re-embedding workflows the finite-height guarantee is designed to cover.

2.6 Reference checking

At the fixpoint—and only if the machine converged (Lemma 4)—the analysis collects, for each state s with entry $\Phi(s) = \langle \sigma_d, \sigma_v \rangle$ and effective input $\text{eff} = \text{narrows}(\sigma_d, \text{InputPath})$, the read sites at which the runtime evaluates a reference and aborts when it is absent. There are three families.

- *JSONPath payload reads.* The $.\$$ keys of **Parameters** and **ItemSelector** (at any depth) and a **Map**’s **ItemsPath** string, resolved by $\text{res}(\cdot, \langle \text{eff}, \sigma_v \rangle)$: a document path against the effective input, a variable reference against the variable store.
- *JSONPath Assign reads.* The $.\$$ values of the state’s own **Assign** block, resolved by $\text{res}(\cdot, \langle \sigma_s, \sigma_v \rangle)$ against the state-dependent **Assign** source. A **Catch** rule’s own **Assign** block is *not* a checked site in this mode, although it does contribute to the transfer: its source document is the opaque error output, so a document read there is never definitely absent, and we do not check its variable reads either—a deliberate abstention, and a completeness gap rather than a soundness one.
- *JSONata reads.* For a state in JSONata mode, the expressions appearing in **Arguments**, **Output**, **Assign**, **Items**, each **Choices** rule’s condition, and each **Catch** rule’s **Assign**,

resolved against $\mathcal{S}(\Phi(s), s)$ —but *only* those whose entire body is a bare direct reference (Assumption 2).

Excluded from all three are the references the analysis does not interpret: any string not beginning with \$, the context object \$\$, `States.*` intrinsics, the bare root \$, and any path outside the simple-path fragment. For a surviving read site with path p resolving against component σ , `STEPCHECK` reports `SC1101` iff p is definitely absent in σ (the lookup returns `Missing`; a path into a $\top$ or otherwise unconstrained region returns `Maybe` and is never flagged). The check *does not* inspect `Choice` operands in `JSONPath` mode; see Section 2.8.

Assumption 1 (Runtime resolution on non-objects). The ASL `JSONPath` engine resolves a dotted reference $k_1 \dots k_m$ by successive object-key lookup: applying a key to a non-object (scalar or array) fails with a path-not-found error, and a numeric segment denotes a property name rather than an array index [11, 9]. Under this assumption the concrete *present* relation of Section 2 (object-key descent) coincides with runtime resolution, and γ ’s admission of arbitrary non-objects under `Obj` r is sound: a reference descending into a non-object simply fails, exactly as “absent” predicts. The same holds for a variable reference, whose first segment selects a binding from the store and whose remainder descends the bound value.

Assumption 2 (JSONata undefined propagation). In the JSONata dialect ASL implements, a path expression that matches nothing evaluates to `undefined` rather than raising. ASL fails the state with `States.QueryEvaluationError` when a field’s value expression evaluates to `undefined` at the top level (a “failure to return a result”), but a JSONata *object constructor omits* a key whose value is `undefined`, yielding a smaller object rather than an error [11]. Only an expression whose *entire* body is a bare direct reference is therefore a hard-failing read site; a reference nested inside a constructor is not, and flagging one would be a false positive. The analysis accordingly tracks a constructor’s *shape* while checking only the bare sites—the distinction this assumption forces.

Theorem 1 (No false positives). *If `STEPCHECK` reports `SC1101` for a read site with path p at a state s , then in every concrete execution that reaches s , p is absent from the value that read site resolves against—the state document for a `JSONPath` document reference or `$states.input`, the variable store for a variable reference, and the state’s result or error output for the corresponding JSONata source—so the ASL runtime fails the reference (Assumptions 1 and 2). Equivalently, the analysis never flags a reference some execution could satisfy.*

Proof. First, the fixpoint over-approximates the reachable environments: for every s and every $(d, \nu) \in \llbracket s \rrbracket$, $(d, \nu) \in \Gamma(\Phi(s))$. We argue by induction on the length of the execution prefix reaching s . *Base:* the start seed $\varepsilon_0 = \langle \sigma_0, \text{Obj } \emptyset \rangle$ satisfies $\llbracket s_0 \rrbracket \subseteq \Gamma(\varepsilon_0)$. On the document component, in the native tier $\sigma_0 = \top$ and $\gamma(\top) = \mathcal{D}$; in the typed tier $\sigma_0 = \text{Obj } r_0$ with $\text{dom}(r_0)$ the declared input fields, and the claim holds under the closed-world reading of the declared contract (the input contains no field outside the schema). On the variable component the seed is the *empty* record and the claim is unconditional: ASL introduces workflow variables only through `Assign`, so at the start state the store is empty and $\text{obj}(\nu_0) = \{\} \in \gamma(\text{Obj } \emptyset)$. *Step:* suppose s is reached by a transition from s' . If it is a normal transition, the concrete environment is $\text{out}_c(s', (d', \nu'))$ for some $(d', \nu') \in \llbracket s' \rrbracket$; by the induction hypothesis $(d', \nu') \in \Gamma(\Phi(s'))$, by Lemma 3 $\text{out}_c(s', (d', \nu')) \in \Gamma(\text{out}(s', \Phi(s')))$, and since the round joined $\text{out}(s', \Phi(s'))$ into $\Phi(s)$, by Lemma 2 $\text{out}_c(s', (d', \nu')) \in \Gamma(\Phi(s))$. If it is a `Catch` transition, the concrete document is the input merged with an (arbitrary) error object and the store is the pre-failure store, possibly extended by the catcher’s `Assign`; *catch* places $\top$ at the catch path, which admits every such document, and applies *asg* with source $\top$ on the store, sound by Lemma 3; the round joined the result into $\Phi(s)$, so again the environment lies in $\Gamma(\Phi(s))$. The `InputPath` narrowing applied at s is itself sound (Lemma 3), so the *effective* entry documents lie in $\gamma(\text{eff})$; and the four JSONata

sources over-approximate their runtime values, since `input` is σ_d , `vars` is σ_v , `errorOutput` is $\top$, and `result` is either $\top$ or a declared/known-envelope shape the task’s result inhabits under the same closed-world reading as the typed seed.

Now suppose SC1101 fires for a read site with path p at s , and let σ be the component it resolves against— eff , σ_v , σ_s , or a JSONata source. Then p is definitely absent in σ , and by Lemma 1 no value in $\gamma(\sigma)$ contains p . Every concrete value that read site resolves against at s lies in $\gamma(\sigma)$ —by the over-approximation just established, together with Definition 6 for the variable component—so p is absent on all of them. By Assumption 1 a JSONPath read of p then fails on every execution reaching s , and by Assumption 2 a bare JSONata read of p evaluates to `undefined` and fails the state. A finding is emitted only when the machine converged (Lemma 4), i.e. at a genuine fixpoint, so the conclusion holds whenever SC1101 is reported. $\square$

Corollary 1 (Native unconditionality). *On the native tier ($\sigma_0 = \top$), Theorem 1 holds for an arbitrary execution input. On the typed tier it holds relative to the closed-world reading of the declared input schema; where inputs are open, the native tier is the sound fallback. (The typed seed also admits a second closed-world source: when a workflow declares no input schema but its start task declares an input contract, the start seed is that contract’s field record.) The variable component is unconditional in both tiers, because its seed—the empty store—is fixed by ASL rather than declared: a read of a variable that no reachable `Assign` binds is a definite fault on any input.*

2.7 Nested machines

A `Map`’s iterator and a `Parallel`’s branches are themselves state machines, each analysed by the same procedure under a seed derived from the enclosing state’s entry.

Remark 2 (Nested-machine seeds). The nested analysis runs—crucially for soundness—only when the enclosing machine has converged (Lemma 4); otherwise its entry shapes are under-approximate and the recursion is suppressed together with the enclosing machine’s own reporting. Each `Map` iteration is seeded with $\text{ctor}(\text{ItemSelector})$ when an `ItemSelector` is declared and $\top$ otherwise; each `Parallel` branch is seeded with $\text{ctor}(\text{Parameters})$ when the state declares `Parameters` and with the effective input $\text{eff} = \text{narrows}(\sigma_d, \text{InputPath})$ otherwise—exactly the document ASL hands each branch. The variable component is inherited unchanged by a `Parallel` branch and by an inline `Map` iteration, which run inside the parent execution and see its bindings; a *distributed* `Map` is seeded with $\top$ instead, because each item runs as a separate child execution that does not inherit the parent’s variables, and assuming otherwise would be unsound. Each seed over-approximates the corresponding per-iteration or per-branch environment (Lemma 3), so Theorem 1 extends unconditionally to findings emitted inside a nested machine.

2.8 The dead-guard refinement (SC1110)

The same shapes support a path-sensitive *warning*. Unlike a `.$` payload reference, a `Choice` comparator is a *guard* evaluated in rule order: the runtime tries each rule until one matches and follows its branch. A value comparator on an unresolvable operand does not silently return false—ASL raises a `States.Runtime` error [11], which is precisely why `IsPresent` exists. We nonetheless report only a *warning*, not an SC1101 error, for two reasons: the rule may never be reached (an earlier rule can match first and divert control), so we cannot promise a runtime abort; and the finding’s real content is that a branch whose guard *requires* a definitely absent field can never be taken—dead logic. SC1110 fires when a `Choice` rule’s condition requires presence of a field definitely absent in the state’s effective (post-`InputPath`) shape. Presence is required by value comparators, `IsNull`, the type predicates, and `IsPresent:true`; it is *not* required by `IsPresent:false`. The check is conservative on Boolean structure: a conjunction is dead if any

Table 2: Field-read coverage by the modeled JSONPath fragment (`stepcheck path-coverage`). “Doc. reads” are the genuine document-field references (dotted + whole-\$ + complex), excluding the \$\$ context, the `States.*` operands and \$-variable references: the first two are not document references and cannot miss a field, and the third resolves against the variable store rather than the document. “Precise” is dotted + whole-\$.

Corpus	Reads	Dotted	Whole-\$	Complex	\$var	\$\$	States.*	Precise / doc. reads
AWS (193)	1168	596	146	61	0	225	140	742/803 = 92.4%
CNCF (66)	17	15	0	0	0	0	2	15/15 = 100%
Independent (95)	420	296	57	17	0	20	30	353/370 = 95.4%
Combined	1605	907	203	78	0	245	172	1110/1188 = 93.4%

conjunct is dead, a disjunction only if *all* disjuncts are dead, and a negation is never flagged. Soundness of the underlying “definitely absent” judgment is exactly Lemma 1.

2.9 Coverage of the modeled fragment

The analysis resolves precisely the reads inside the simple-path fragment (\$, \$.a, \$.a.b); every operand outside it lifts to $\top$ and can never produce a finding. To quantify how much of real ASL this covers, the `stepcheck path-coverage` command classifies every JSONPath field-read a workflow evaluates—the \$. operands of `Parameters/ItemSelector`, a `Map’s ItemsPath`, and the \$. operands of `Assign`, i.e. the same read sites the SC1101 check consumes in JSONPath mode—into the dotted fragment, the whole-document \$, complex JSONPath (bracket/wildcard/filter/function), the \$\$ context object, \$-variable references, and `States.*` intrinsics. Table 2 reports the split over the AWS corpus, the CNCF examples, and the independent repositories of Section 2.10.

Of the genuine document-field references, 92–95% fall inside the modeled dotted fragment; only 6.6% (combined) use bracket, wildcard, or filter syntax that the analysis soundly abstracts to $\top$. The CNCF corpus contributes few reads because Serverless Workflow expresses data movement as runtime expressions rather than as a JSONPath pipeline; the 15 jq field reads the front end does lower all fall inside the dotted fragment, and the ASL-specific pipeline checks have little to act on there. Workflow-variable references are resolved—against the variable store—rather than abstracted, but no \$. read site in these corpora is one, so the JSONPath split is identical with and without the variable domain. That domain earns its place on workflows that read what they bind: a read of a name no reachable `Assign` binds is a definite fault the document analysis alone cannot see (Corollary 1).

2.10 External validity: independent repositories

Beyond the three aws-samples collections that make up the main corpus, we ran STEP CHECK unmodified on **95** ASL definitions mined (`eval/mine_wild.js`) from **16** public repositories outside them—50 domain workflows across 13 repositories, the remainder validator/library fixtures. Two findings from the sound and conservative native tiers illustrate that the analyses are not overfit to AWS’s samples.

A sound SC1101 in the wild. In a credit-card *transaction-reversal* workflow, a `Map` state `ApplyReversalEntries` iterates `ItemsPath`: \$.`ledgerEntries`. The start state `SetReversalDefaults` is a `Pass` whose `Parameters` reconstruct the document as a closed record of seven fields (`transactionId`, `amount`, `currency`, `accountId`, `reason`, `reversalId`, `reversalStatus`); `ledgerEntries` is not among them, and the two intervening states merge only their own results under \$.`validation` and \$.`screening`. By the transfer functions of

Section 2, the shape at `ApplyReversalEntries` is a closed record with `ledgerEntries` $\notin$ dom, so `present($ledgerEntries)` is definitely false and SC1101 fires soundly—at run time the Map fails to resolve `ItemsPath`. Because `SetReversalDefaults` *reconstructs* the document rather than augmenting it, even a caller that supplies `ledgerEntries` has it dropped; an adjacent-task field-subset check, which never models the `Parameters` reconstruction, cannot see this.

A conservative SC5001 in the wild. In an IoT firmware-upgrade workflow, a `Parallel` state runs three branches that each write the DynamoDB table `FWUTaskTokens` (`PutItem`); the branches receive document copies, so the shared external table is the only race, and the concurrency pass reports the provable last-writer-wins collision.

Declared-tier evidence. The declared tier is silent on the wild corpus only because no workflow ships a compensator. Supplying the missing facts recovers it: for an independent payment-settlement pipeline, a sidecar declaring the fraud flag, the payment authorization and the settlement as persistent effects (all are durable money movements) promotes SC4001 on the uncompensated `FlagFraudulentPayment` from an inferred warning to a declared-tier *error*. SC4010 on `AuthorizePayment/SettlePayment`, whose `Catch` paths only reach a `Fail` state (so a settlement failure leaves an authorized hold that is never voided), stays a warning: that check is warning-level whatever the provenance of its premise. The sidecar ships in the artifact (`corpus/wild-annot/`); the workflow itself comes from a repository without a licence, so it is not redistributed and is fetched from upstream through `corpus/wild-external/manifest.json`.

3 The workflow type discipline

3.1 Signatures, judgment, and validity

Let Γ collect the declared task signatures. Each task T has a *contract* $A \rightarrow B$ (it consumes the field-record A and produces B) and a *typestate* [7, 8] transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ over business states S_B ; we bundle the two into one signature $T : (A, s_{\text{in}}^T) \rightarrow (B, s_{\text{out}}^T)$. The well-typedness judgment $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ carries a *pair* type—a field-record and a business state—and is generated by:

$$\frac{(T : (A, s_{\text{in}}) \rightarrow (B, s_{\text{out}})) \in \Gamma}{\Gamma \vdash T : (A, s_{\text{in}}) \Rightarrow (B, s_{\text{out}})} \text{ (TASK)} \quad \frac{\Gamma \vdash T : (A, s) \Rightarrow (B, s') \quad A \subseteq A'}{\Gamma \vdash T : (A', s) \Rightarrow (B, s')} \text{ (SUB)}$$

$$\frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (B, s') \Rightarrow (C, s'')}{\Gamma \vdash T_1; T_2 : (A, s) \Rightarrow (C, s'')} \text{ (SEQ)}$$

$$\frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (A, s) \Rightarrow (B, s')}{\Gamma \vdash \text{Choice}(T_1, T_2) : (A, s) \Rightarrow (B, s')} \text{ (CHOICE)}$$

The width-subsumption rule (SUB) widens the *available* input fields—a task needing A still runs when more fields $A' \supseteq A$ are present—and leaves the business state untouched. It is what lets the field component be matched by *containment* while the typestate component is matched by *equality*, mirroring the two checks below; without it the SEQ rule would force the middle field-record to match exactly, which the tool does not require.

The judgment is read over the workflow’s *task-adjacency* structure: `STEPCHECK` projects the control-flow graph onto its `Task` states, following *normal* edges through the *transparent* states (`Choice`, `Pass`, `Wait`) that carry no contract, so $T_1; T_2$ denotes a task immediately followed—through any transparent states—by another and `Choice`(T_1, T_2) a branch whose arms rejoin. `Catch` (error) edges are *not* projected, and `Map/Parallel` states are opaque adjacency endpoints whose sub-machines are checked separately, so the discipline governs the normal-flow task

graph only. A declared *protocol* $\delta \subseteq S_B \times S_B$ lists the legal business-state transitions, and $\text{Valid}(W) \iff \forall T \in W. (s_{\text{in}}^T, s_{\text{out}}^T) \in \delta$.

The checks are edge-local. For each task-adjacency $T_1 \rightarrow T_2$ whose endpoints both carry the relevant declarations: the typed-contract check SC1010 fires iff some required field of T_2 is missing from T_1 's produced fields ($\text{in}(T_2) \not\subseteq \text{out}(T_1)$); the typestate check SC2001 fires iff $s_{\text{out}}^{T_1} \neq s_{\text{in}}^{T_2}$; and SC2002 fires iff $(s_{\text{in}}^{T_2}, s_{\text{out}}^{T_2}) \notin \delta$ when a non-empty protocol is declared. A **Choice** imposes the predecessor's premise on each branch head. An edge missing an endpoint declaration is passed over—the checks are vacuous there—so the guarantee below is stated for *fully declared* workflows.

Theorem 2 (Declared-tier consistency). *Let W be fully declared: every **Task** carries a contract and both typestate labels, and a protocol δ is declared. Then SC1010, SC2001, and SC2002 emit no finding iff*

$$\begin{aligned} & \text{in}(T_2) \subseteq \text{out}(T_1) \text{ and } s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2} \text{ for every adjacency } T_1 \rightarrow T_2, \\ & \text{and } (s_{\text{in}}^T, s_{\text{out}}^T) \in \delta \text{ for every task } T. \end{aligned} \quad (*)$$

The three checks are therefore a sound and complete decision procedure for (): every W satisfying (*) is accepted, and every violation is reported by a finding naming the offending edge (SC1010/SC2001) or task (SC2002). On the series-parallel fragment—acyclic task graphs built from sequencing and fully rejoining binary **Choice** whose arms share a common branch-point and rejoin pair type—condition (*) is moreover equivalent to derivability $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ together with $\text{Valid}(W)$, so on that fragment the checks decide well-typedness in the calculus above.*

Proof. The equivalence “no finding $\iff$ (*)” is immediate: each check is exactly the negation of one clause of (*), evaluated independently per edge (per task for SC2002), and full declaration leaves no edge vacuous. This holds for an arbitrary normal-flow task graph, cyclic ones included—a back-edge is simply another adjacency subject to (*).

For the series-parallel fragment we show $(*) \iff \Gamma \vdash W : (A, s) \Rightarrow (B, s') \wedge \text{Valid}(W)$. ($\Leftarrow$) A derivation types each adjacency by SEQ, whose shared middle type (B, s') forces $\text{in}(T_2) \subseteq \text{out}(T_1)$ on the field component (a preceding SUB step only enlarges the available fields) and $s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2}$ on the typestate component; CHOICE imposes the same shared type on both arms, extending the equalities across a branch and its rejoin. $\text{Valid}(W)$ supplies the protocol clause, giving (*). ($\Rightarrow$) Assume (*). Build the derivation bottom-up: TASK gives each task $T : (\text{in}(T), s_{\text{in}}^T) \Rightarrow (\text{out}(T), s_{\text{out}}^T)$. For a sequential adjacency $T_1; T_2$ take the middle type $(B, s') = (\text{out}(T_1), s_{\text{out}}^{T_1})$: by (*), $\text{in}(T_2) \subseteq \text{out}(T_1) = B$ and $s_{\text{in}}^{T_2} = s_{\text{out}}^{T_1} = s'$, so SUB lifts the TASK axiom of T_2 to $(B, s') \Rightarrow (\text{out}(T_2), s_{\text{out}}^{T_2})$ and SEQ applies. A fragment **Choice** types both arms at the common branch-point type (A, s) and common rejoin type (B, s') (each arm as above), so CHOICE applies. The result derives $\Gamma \vdash W$, and the per-task protocol membership gives $\text{Valid}(W)$. Outside the fragment—non-rejoining or multiway branches, type-misaligned rejoins, or cycles—no finite derivation in this three-constructor calculus need exist even when (*) holds; there the edge-condition statement is the operative one, and it is exactly what the tool decides.

Both readings are *relative to the declared annotations*, and here that scoping is exact: contracts and typestate labels are never inferred—only the idempotence/persistence/effect annotations of Section 4 are—so on this tier Γ is the declared data and the three checks are unconditionally error-level. A workflow lacking a declaration is outside the theorem's scope (the corresponding check is vacuous on that edge), not silently downgraded to a warning. $\square$

4 Downstream-aware Saga compensation

4.1 Model of failure and the post-commit region

Let W declare which tasks are *persistent* and, for each persistent P , a declared *compensator* C (the Saga pattern [6]). Every *fallible* state—a `Task`, `Map`, or `Parallel`—may raise the generic error `States.TaskFailed`; an error caught by a *broad Catch* (one whose `ErrorEquals` contains `States.ALL` or `States.TaskFailed`) is routed to the catch target, otherwise it propagates to workflow failure. `Parallel` and `Map` are fallible black boxes; a nested persistent task is analysed within its own sub-machine by the same procedure.

The completeness half of the guarantee below is stated relative to an explicit model of execution, made precise here so the theorem is neither over- nor under-claimed.

Assumption 3 (Failure model). *(A1) Single post-commit fault:* on each execution we reason about the *first* failure after P commits, and the compensation flow it triggers is itself fault-free—a standard single-fault Saga assumption. *(A2) Path feasibility:* `Choice` guards are abstracted, so every normal edge is feasible (the analysis is path-insensitive). *(A3) Unique generic catcher:* no fallible state has two `Catch` clauses both matching `States.TaskFailed`, so the existential test over catchers coincides with ASL’s first-match resolution for the modeled error. *(A4) Linear handlers:* from a broad `Catch` target, control follows a unique normal successor until it reaches C (as the typed DSL builds compensation chains), so “reaches C ” means “executes C ”.

Definition 7 (Post-commit region). $R(P)$ is the set of states reachable from P ’s normal successors by following *normal* (non-error) control-flow edges *without passing through* C . These are exactly the states that execute after P has committed its durable effect and before any compensation.

Definition 8 (Saga-completeness). A persistent task P with compensator C is *Saga-complete* when every execution in which P commits its effect and a subsequent *post-commit failure* occurs—a fallible state in $R(P)$ raising the generic error, or a `Fail` terminal reached on a normal path in $R(P)$ —also executes C .

4.2 The SC4011 algorithm

Listing 1: Downstream-aware Saga check (per persistent task).

```
SC4011(W, P, C):
  region = post_commit_region(W, P, C)          # the region R(P)
  foreach Q in region:
    if is_fallible(Q) and not failure_reaches(Q, C):
      fire SC4011 at Q                          # a post-commit failure at Q
    escapes C
    if is_fail_terminal(Q):
      fire SC4011 at Q                          # a normal path commits P then ends
    in failure

post_commit_region(W, P, C):                    # normal-flow closure of P's
  successors,                                  #
  return forward_closure(normal_successors(P),  # not passing through C
                        edges = normal, without = {C})

failure_reaches(Q, C):
  # Q's own generic failure is caught broadly and the handler reaches C by normal
  flow
  return exists c in catches(Q):
    broad(c.error_equals) and reaches(c.target, C, edges = normal)
```

```

is_fallible(Q):      return Q.kind in {Task, Map, Parallel}
is_fail_terminal(Q): return Q.kind == Fail
broad(errs):         return "States.ALL" in errs or "States.TaskFailed" in errs
reaches(s, C, e):    return C in forward_closure({s}, edges = e)

```

Theorem 3 (Compensation completeness under the declared model). *Fix a workflow W with declared persistence and compensators, and adopt Assumption 3. For a persistent task P with compensator C , SC4011 (Listing 1) emits no finding iff P is Saga-complete—iff every fallible $Q \in R(P)$ has a broad **Catch** whose target reaches C , and no normal path from P reaches a **Fail** terminal in $R(P)$. Under the model, SC4011 is a sound and complete decision procedure for Saga-completeness.*

Proof. Soundness (every finding is real, relative to A2). Suppose SC4011 fires. In the first case it fires at a fallible $Q \in R(P)$ with $\neg \text{failure_reaches}(Q, C)$. By A2 the normal path $P \rightarrow \dots \rightarrow Q$ is feasible, so some execution runs P to commitment and reaches Q ; being fallible, Q raises **States.TaskFailed** (A1: the single post-commit fault). By A3 the catcher the test inspects is the one ASL would match, and it is absent, non-broad, or routes to a handler not reaching C ; by A4 “reaches C ” is exact, so C does not execute. This is a concrete post-commit failure that leaks P ’s effect. In the second case SC4011 fires at a **Fail** terminal $Q \in R(P)$: the feasible normal path $P \rightarrow \dots \rightarrow Q$ commits P and then fails with no compensation—a more direct witness.

Completeness (every defect is found). Suppose P is *not* Saga-complete: some execution commits P and then a post-commit failure occurs without C . By A1 consider that first failure. If it is a fallible state Q , then $Q \in R(P)$; by A3 its generic error resolves to the single catcher the test inspects, and since C did not run, that catcher does not route to C (A4 makes “reaches” exact), so $\neg \text{failure_reaches}(Q, C)$ and SC4011 fires at Q . If instead the execution reaches a **Fail** terminal on a normal path without C , that terminal lies in $R(P)$ and SC4011 fires there. Conversely, if every fallible $Q \in R(P)$ has its (unique, by A3) broad catcher reaching C and no normal path bypasses C to a **Fail**, then by A4 every post-commit fault routes into a chain that runs C , so P is Saga-complete. Hence SC4011 fires exactly on the non-Saga-complete workflows.

The guarantee is *relative to the declared facts*: a compensator is never *inferred*, so under pure inference the premise is absent and SC4011 stays silent (in contrast to the name-based, warning-level SC4010). $\square$

Remark 3 (Where the assumptions bite). Relaxing Assumption 3 costs completeness, not soundness. Without A1, a fault *inside* the compensation chain (e.g. a fallible compensator) can leak an earlier effect, which SC4011 does not model. Without A3, a fallible state whose first catcher matches **States.TaskFailed** but routes away from C , shadowing a later broad catcher, is missed. Without A4, a broad catcher whose handler *may* reach C on one branch and a **Fail** on another passes the existential test yet can skip C ; deciding this exactly needs C to *postdominate* the handler, which the check approximates by reachability. Non-generic errors (**States.Timeout**, service faults) caught by narrow catchers, and cross-branch aborts of a **Parallel/Map** that strand a sibling’s committed effect, are likewise outside the model. Closing these is future work; the corpus in the main paper satisfies A1–A4. Finally, the theorem presupposes *declared* persistence and compensators: where a workflow declares neither, SC4011 is silent altogether and compensation coverage falls back to the per-task checks SC4001/SC4010, which run on inferred premises at warning level and see only whether a persistent task has some compensating path, not whether a later post-commit failure bypasses it.

4.3 Reverse-order chains

Saga-completeness of a single (P, C) pair is local, but a multi-step Saga must unwind *all* prior effects on a late failure. Take persistent tasks $P_1, \dots, P_n$ committed in order with compensators

$C_1, \dots, C_n$ wired as a *reverse-order chain*: each C_i 's *unique* normal successor is C_{i-1} , and C_1 's is the failure terminal. Under this linear-chain hypothesis—exactly A4 of Assumption 3 applied to the chain—running C_i runs the whole prefix $C_i, C_{i-1}, \dots, C_1$ in order, so applying the single-pair check of Theorem 3 to each (P_i, C_i) certifies that any post-commit failure unwinds the full committed prefix. The hypothesis is essential: without the unique-successor wiring, per-pair reachability certifies neither the *order* nor the *completeness* of the unwind—a branch in the chain could skip some C_j while still reaching a later C_k . This is how the typed DSL builds a Saga by construction, which is why a correctly built DSL Saga raises no SC4011 and a one-edge break (redirecting one compensator to the failure terminal) makes SC4011 fire on precisely the upstream steps whose effects would then leak (the chain-break experiment in the main paper's evaluation).

5 Conservative analyses: concurrency and temporal (notes)

The concurrency and temporal analyses are native but deliberately *conservative*: they under-approximate to avoid false alarms and make no soundness claim. All but one of their findings are warnings; the sole exception is the definite mis-configuration SC6003 (below), reported as an error.

Concurrency (SC5001/SC5002). Parallel branches and concurrent Map iterations each receive their own copy of the document, so the document cannot race; only *external* resources can. The check mines each task's resource key—service plus the named target: `TableName`, `QueueUrl`, `TopicArn`, `Bucket`, `EventBusName`, ECS cluster/task identifiers, `Bedrock ModelId`, and nested-workflow `StateMachineArn/Name` pairs (statically named `states:startExecution` calls that provably share a child workflow and execution name also collide)—and considers only *overwriting* writes—`PutItem`/`PutObject`, which are last-writer-wins. It does not inspect a separately defined child workflow unless that child definition is checked as a workflow of its own. A task counts as a write only when it is annotated non-idempotent (or persistent): an unannotated `PutItem` is not flagged, a deliberate precision trade that keeps the analysis silent absent an effect declaration and contributes to its zero-false-positive record. The check also excludes `UpdateItem` (an atomic merge that commutes for disjoint attributes) and fire-and-forget `Publish`/`SendMessage` (independent messages). SC5001 flags two Parallel branches that overwrite the same resource; SC5002 a concurrent Map (`MaxConcurrency` $\neq 1$) whose iterations overwrite a shared resource while threading *no* per-item data (no `.$` reference in the write, so every iteration clobbers the same item). Restricting to overwriting writes with a provably shared key is what makes the analysis report a conflict only when two branches *provably* touch the same key—hence zero in-the-wild false positives, at the cost of not claiming completeness.

Temporal (SC6001–SC6003). SC6001 warns when a callback (`waitForTaskToken`) or Activity task has neither `TimeoutSeconds` nor `HeartbeatSeconds`, so it can wait up to the platform's one-year maximum. SC6002 warns when a task's worst-case retry budget—the geometric back-off sum $\sum_{k=0}^{a-1} interval \cdot backoff^k$ over a attempts (defaults `interval` = 1s, `backoff` = 2, a = 3)—exceeds a declared machine `TimeoutSeconds`. SC6003 is a definite mis-configuration: `HeartbeatSeconds` $\geq$ `TimeoutSeconds`, which the ASL specification forbids but AWS's definition validator does not reject (it misses all 166 injected SC6003 mutants). SC6001 is a warning because a missing timeout is occasionally acceptable for a short-lived task; SC6003 is the one error-level temporal finding, as it is unambiguous.

Remark 4 (What the data-flow abstraction still lifts to $\top$). Section 2 covers the state document, `Assign`-bound workflow variables, and the JSONata fragment of bare source references and object constructors. Four families remain opaque, each soundly: JSONata expressions outside

that fragment (arithmetic, function calls, predicates, string operations); JSONPath outside the simple-path fragment (wildcards, filters, brackets, array indexes); the context object `$$` and `$states.context`, which are runtime metadata rather than workflow-produced data; and array *elements*, since an array abstracts to `T`. Each lift can only suppress a finding, never create one (Lemma 4), and the coverage measurement of Section 2.9 bounds what the first two cost in practice. Refining the array case to an element shape, so that a `Map` iterating a constructed array carries its item record, is the natural next step.

6 Artifact and implementation notes

This section records the implementation and reproduction detail deferred from the main paper’s implementation and artifact sections. The artifact contains the verifier, the corpora used in the paper, the mutation engine, the concrete execution oracle, the validator-panel scripts, the timing harness, the real-bug replay harness, and the scripts that regenerate the main paper’s figures and tables. The AWS corpus has 193 Step Functions workflows; the cross-format corpus has 66 CNCF Serverless Workflow examples.

Excluding blank lines, comments, and tests, STEP CHECK is 6,160 lines of Rust, divided as follows: the workflow IR, 262 lines; the four frontends (ASL parse and emit, the typed DSL, CNCF [10], and the CloudFormation/SAM/CDK template reader), 1,128; annotation resolution and inference, 324; the eight analysis passes and their manager, 2,327; diagnostics, 99; the execution oracle, 373; the mutation engine, 485; and the command-line interface together with the statistics, evaluation, and oracle harnesses, 1,162; a further 879 lines of in-tree tests are not counted above. The data-flow pass is by far the largest analysis at 1,316 lines—the shape lattice of Section 2, its transfer functions, and the control-flow fixpoint—while the other seven passes range from roughly 66 to 269 lines each. (These counts are for the artifact revision and drift with continued development.)

Packaging and distribution. The artifact ships a continuous-integration pipeline that builds and tests the binary on Linux, macOS, and Windows on every change, and a release pipeline that packages it for language package managers—a `crates.io` crate and an `npm` wrapper package that fetches the platform binary—so a Rust- or Node-centric CI adopts it with one command; a copy-pasteable GitHub Actions gate is included. With `-json`, findings emit one stable object per finding carrying a code, severity, offending state, message, and remediation note (e.g. SC1101 at ChargeCard: “reads `$total`, a field no execution reaching it can have produced,” noting that no state on any path binds it).

The command-line tool exposes `check`, `infer`, `scan`, `stats`, `eval`, `oracle`, `dataflow-cert`, `eval-pairs`, `mutate`, `fixpoint-stats`, `path-coverage`, `emit`, and `demo`. `check` runs the eight passes; `emit` round-trips a workflow through the ASL emitter unchanged, the control form against which the multi-validator baseline of the main paper scores each injected defect; `oracle` runs the data-flow soundness cross-check; `dataflow-cert` re-validates the data-flow proof-certificate obligations; `fixpoint-stats` reports fixpoint round/bound utilisation; `path-coverage` measures the fraction of document-field references inside the modeled fragment; `eval-pairs` drives the real-bug benchmark by replaying each pre-fix and post-fix workflow; `mutate` injects one defect of a chosen class; and `demo` emits a built-in typed-DSL example to ASL.

The evaluation is reproducible offline on a commodity laptop. A Rust 1.96 toolchain builds the verifier and runs the mutation study, data-flow oracle, timing experiments, and real-bug replay. Node.js and Ruby drive the two local third-party validators used in the baseline (`asl-validator` and `statelint`), and Python renders the figures. No AWS account or credentials are required to reproduce the paper’s tables, figures, mutation results, data-flow oracle, timing results, or local validator baselines. AWS’s server-side `ValidateStateMachineDefinition` check is opt-in (`STEP CHECK_AWS=1`), creates no resources, and has recorded outputs committed with the artifact

so its numbers reproduce offline. The only step that touches an AWS account is the optional live deployment test reported in the main paper (Express, Standard, and callback machines); no other result depends on it.

The ASL emitter is currently lossy for a workflow’s pure I/O-processing fields: among others it does not re-emit `InputPath`, `OutputPath`, or `ResultSelector`, nor the machine-level `TimeoutSeconds`; it also drops the `QueryLanguage` directive (a JSONata machine re-emits without it) and canonicalises a `Map`’s `ItemProcessor` to the older `Iterator` form. The AWS round-trip is therefore exercised on DSL-authored workflows that use none of these fields, where the deployed state machine is behaviourally equivalent to the verified IR.

7 Baselines: the Asl-to-BPMN encoding and tool configuration

The main paper compares STEP CHECK with two families of baselines. The schema validators run on the ASL definition directly. The workflow-soundness verifiers decide a control-flow property over Petri-net or BPMN models, so each workflow must first be encoded. This section records that encoding and the exact configuration of every baseline, which the paper cites but has no room for.

7.1 The encoding

The encoder (`eval/asl2bpmn/encode.js`) is dependency-free and scoped to the control-flow fair class:

ASL construct	BPMN
<code>Task / Pass / Wait</code>	activity
<code>Choice</code>	exclusive gateway, one guarded flow per rule plus <code>Default</code>
<code>Parallel</code>	parallel gateway split ... branches ... parallel gateway join
<code>Map</code>	activity placeholder (item data flow out of scope)
<code>Catch</code>	exclusive gateway after the task: normal completion vs. one branch per handler
<code>Retry</code>	lossy metadata (no control-flow change)

Terminals. Every terminal—`End: true`, `Succeed`, `Fail`, or a state with no successor—is routed to the single end event of its scope: the branch exit inside a `Parallel`, the shared end event at the top level. Each encoded workflow is therefore a workflow net (one source, one sink) *by construction*, which is what a classical soundness checker requires. A dangling `Next` (for instance after a structural mutation) is materialised as a dead-end placeholder so that the net still builds and the verifier genuinely reports the defect, rather than the encoder failing to parse and the mutant being silently credited.

What the over-reports are, and are not. Because terminals are merged, the workflows that the verifiers call unsound on the clean corpus are *not* an artifact of multiple sinks. The harness records the residual cause as parallel-branch termination: workflows whose `Parallel` branches or error exits can terminate independently of their siblings, which classical proper completion forbids. The rates are Woflan 16/193 (8.3%), BPMN Analyzer 16/193 (8.3%) and BProVe 56/193 (29%). These figures are properties of *this* encoding together with the classical soundness definition, not verdicts on the tools, and we did not evaluate alternative encodings of branch and error exits, which would likely change them.

What is not encoded. JSON document provenance, task contracts, service semantics, executable guards (**Choice** conditions survive only as flow labels), retry counts and **Map** item data flow are all outside the encoding. In particular SC1101 has no BPMN control-flow expression at all, which is why the data-flow class is evaluated on the typed tier rather than in the head-to-head.

7.2 Crediting rule and tool configuration

For every workflow and class k , the control is `stepcheck emit` (a canonical re-emit) and the mutant is `stepcheck mutate -kind k`, so the two differ by exactly one injected defect. A verifier is credited with a detection iff the mutant’s encoding carries a soundness violation absent from the control’s; a schema validator is credited iff it emits a fresh problem that names the injected fault, so incidental schema nits do not count. The baselines were configured as follows.

- **Woflan** [2] via `pm4py 2.7.23.1 (pm4py.algo.analysis.woflan)`.
- **BPMN Analyzer 2.0** [3] via the `rust_bpmn_analyzer` CLI, built from source with `cargo build -release`.
- **BProVe/BPMNOS** [4] via the BPMNOS parser and the Maude model checker, checking safeness, option-to-complete and proper-completion.
- **statelint** (AWS Labs, J2119) run under Ruby 3.3, and **asl-validator** (npm), both invoked by `eval/validator_panel.js`.
- **AWS ValidateStateMachineDefinition** is opt-in (`STEPCHECK_AWS=1`); it creates no resources, and its outputs are committed with the artifact so the numbers reproduce offline.

The three workflow verifiers and their harness record the versions above. The two local validators are resolved from the environment, so the harness records their availability rather than a pinned version; pinning them in the artifact’s lockfile is a reproducibility improvement we recommend to anyone re-running the panel.

8 Mutation operators, per-class results, and inference calibration

8.1 Operators

Each mutation introduces exactly one defect of a known class into a real workflow and re-emits ASL, so the mutant is a genuine, deployable-shaped workflow differing from the original by one localized edit. The designed operators are: break a JSONPath data binding (SC1003); redirect a `.$` reference to a field no state produces (SC1101, typed tier); add a broad retry to a non-idempotent task (SC3001); remove the error handling from a persistent task (SC4001); retarget a transition to a non-existent state (SC0002); make two **Parallel** branches write the same resource (SC5001); and set `HeartbeatSeconds ≥ TimeoutSeconds` (SC6003).

The hard suite keeps each defect but places it just past the boundary of what the corresponding analysis can prove, so every miss is a measured boundary rather than a false alarm:

Class	Code	Hard variant and the boundary it probes
Concurrency	SC5001	the shared table is named by an input field ("TableName.\$") instead of a literal: dynamic resource identity, so the interference analysis lifts to $\top$
Temporal	SC6003	the two durations arrive through <code>HeartbeatSecondsPath/TimeoutSecondsPath</code> reference paths, so the constant comparison is unavailable
Data flow	SC1101	the read is re-rooted behind a filter expression, outside the modeled selector fragment
Contract	SC1003	the broken <code>.\$</code> payload sits inside a <code>ResultSelector</code> rather than <code>Parameters</code> (a generalization control: the scan reads every payload template, so this should still be caught)
Retry	SC3001	a genuinely non-idempotent task whose state name and <code>FunctionName</code> are genericized, so the naming heuristic cannot prove non-idempotence
Compensation	SC4001	the persistent task has a <code>Catch</code> , but it routes to a state that only logs, which a presence-only rule accepts
Structural	SC0002	the dangling <code>Next</code> sits inside a nested <code>Parallel</code> branch or <code>Map</code> iterator (a generalization control: the pass recurses, so this should still be caught)

Recall is credited to the analysis *family* rather than to a single code, because a boundary defect may legitimately be reported by a sibling check: crediting only the expected code would understate the tool while overstating how tightly operators and checks are coupled. The compensation row is exactly that case—the effect-aware SC4010 fires where the presence-only SC4001 does not.

8.2 Per-class results

Table 3 gives the hard-mutant outcome per class with exact Clopper–Pearson 95% intervals. Aggregate recall is $328/688 = 0.48$, against $616/616$ on the designed suite.

Change since submission. The submitted paper reported $162/688 = 0.24$. At that time the SC1003 scan read `Parameters`, `Assign` and `Choice` guards but not `ResultSelector` or a Distributed Map’s `ItemSelector`, so the contract hard mutant—the easy operator’s broken `.$` moved into a `ResultSelector`—was missed on all 166 workflows. Release 0.1.5 extends the scan to every payload template, which raises the contract class to $166/166$ and the aggregate to $328/688$; every other class is unchanged. The mutants themselves are the same; only the check changed.

Five of the seven variants are $\top$ -lifts, family credits or inference abstentions, where the analysis declines to report what it cannot prove or a sibling check reports it instead. The other two are generalization controls: the dangling edge hides in a nested sub-machine and the broken payload moves from `Parameters` into a `ResultSelector`. Both classes stay exact, because the

Table 3: Hard-mutant recall per class (family-credited), with exact 95% intervals.

Class	Code	n	detected	recall [95% CI]
Structural	SC0002	42	42	1.00 [.92, 1]
Contract	SC1003	166	166	1.00 [.98, 1]
Retry	SC3001	111	76	0.68 [.59, .77]
Compensation	SC4001 family	56	44	0.79 [.66, .88]
Concurrency	SC5001	21	0	0.00 [0, .16]
Temporal	SC6003	166	0	0.00 [0, .02]
Data flow	SC1101	126	0	0.00 [0, .03]
Total		688	328	0.48

structural pass recurses into sub-machines and the SC1003 scan reads every payload template—Parameters, ResultSelector, a Distributed Map’s ItemSelector and Assign—together with Choice guards.

8.3 Calibration of the name-based inference

Inferred premises never block a build, and this is why. Against a gold set of 23 workflows (137 tasks), labelled in two independent passes and reconciled by adjudication (Cohen’s κ 0.888 for idempotency and 0.988 for persistence), the name-based inference predicts 110 of the 137 tasks and abstains on 27 (coverage 80.3%). Of the predictions, idempotency is correct for 77 (70.0%) and persistence for 96 (87.3%). The weaker idempotency heuristic misreads stable-key DynamoDB writes and cancel/release/refund compensators, which are idempotent in effect, as non-idempotent. Scored differently—per warning rather than per task—40 of the 44 SC3001/SC4001 warnings that land on a gold-labelled task are true positives (91%: 33/37 for compensation, 7/7 for retry). Names alone therefore carry enough signal to be worth surfacing, but not enough to block a build, which is exactly the severity the inferred tier is given.

9 Sidecar syntax for declared premises

Declared facts enter through a TOML sidecar beside an existing ASL definition (the typed DSL emits the same facts by construction). The file has three kinds of block. A [schemas.<Name>] block names a record and lists its fields. A [tasks.<TaskName>] block carries that task’s premises: idempotent and persistent (booleans), compensation (the name of the compensating task), state_in/state_out (its typestate transition), and input_schema/output_schema (its contract). A [[protocol]] entry declares one legal business-state transition. Names are resolved against the workflow and the schema table, and stale references are reported syntactically as SC0011/SC0012, so a sidecar cannot rot silently as the workflow evolves.

The order saga’s complete sidecar is 68 lines; the structure is:

```
[schemas.OrderCreated]
fields = ["orderId", "amount", "items"]

[schemas.OrderReserved]
fields = ["orderId", "amount", "reservationId"]

[tasks.ReserveInventory]
idempotent      = false
persistent      = true
compensation    = "ReleaseInventory"
state_in        = "OrderCreated"
state_out       = "OrderReserved"
```

```

input_schema    = "OrderCreated"
output_schema   = "OrderReserved"

[[protocol]]
from = "OrderCreated"
to   = "OrderReserved"

```

Five schemas, four such task blocks and four protocol edges make up the file: about eight per-task lines over four tasks, plus the shared schema and protocol blocks. That is the entire declared-tier input behind the paper’s saga results; everything else the checks use is read from the ASL artifact itself.

10 Extended limitations

This section expands the threats-to-validity discussion of the main paper’s evaluation.

Corpus composition. The corpus comes from public AWS and CNCF sample repositories and skews small (median 6 states, maximum 33). We do not include proprietary industrial definitions because workflow files are sensitive artifacts: they expose resource names, account topology, and business-process structure, and no anonymization we are aware of preserves the semantic features the analyses consume (JSONPath references, resource keys, task names for inference). The generated chain, fan-out, and nested-**Map** sweeps exercise scale and topology, but they cannot stand in for industrial semantic diversity—richer contracts, deeper Saga chains, and organization-specific naming conventions that inference has not been tuned on.

Deployment smoke test. The AWS deployment test emits DSL-authored workflows and deploys them unmodified, exercising all three execution models: an order workflow deployed as both an Express- and a Standard-mode state machine, and a live `.waitForTaskToken` callback machine driven to completion through its task token. Each is executed to **SUCCEEDED** and torn down (100/100 live runs). It does not exercise Activity workers or injected service-integration failures; those behaviours are covered only by the static analyses and the offline oracle, not by live execution.

Mutation-operator provenance. The designed operators are ours, so the per-class recall on them (Table 3) bounds from above what an independently authored suite would report. The matched *hard* suite is the mitigation, not a cure: it places each fault just past the boundary the analysis can prove and drops aggregate recall from 1.00 to 0.48, which shows the checks are not tuned to the operators—but both suites are still authored by us. The externally evidenced defects of the main paper’s RQ3, mined fix-commit pairs and user-filed issues, are the only findings whose provenance is independent of our design.

Baseline encoding. The comparison against the workflow-soundness verifiers is relative to the ASL-to-BPMN encoding of Section 7. The over-report rates on the clean corpus—Woflan 16/193 (8.3%), BPMN Analyzer 16/193 (8.3%), BProVe 56/193 (29%)—are properties of that encoding together with the classical soundness definition rather than verdicts on the tools, and alternative encodings of branch and error exits remain untested.

Semantic drift. Declared premises are checked for syntactic rot (SC0011/SC0012) but not for truth: a sidecar that says a Lambda is idempotent when its implementation is not will pass the declared-tier checks. Generating the sidecar from code or infrastructure-as-code—the generated-metadata premise source—is the deployment pattern that keeps declared facts in sync; inference is the fallback and always stays at warning level.

References

- [1] N. Földvári and F. Crăciun. StepCheck: Sound Static Verification of Deployed AWS Step Functions Workflows. In *ICSOC*, 2026. To appear. (The main paper this report accompanies.)
- [2] H. M. W. Verbeek, T. Basten, and W. M. P. van der Aalst. Diagnosing workflow processes using Woflan. *The Computer Journal*, 44(4), 2001.
- [3] T. Kräuter, P. Stünkel, A. Rutle, Y. Lamo, and H. König. BPMN Analyzer 2.0: instantaneous, comprehensible, and fixable control flow analysis for realistic BPMN models. In *BPM*, 2024.
- [4] F. Corradini, F. Fornari, A. Polini, B. Re, F. Tiezzi, and A. Vandin. A formal approach for the analysis of BPMN collaboration models. *Journal of Systems and Software*, 180, 2021.
- [5] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.
- [6] H. Garcia-Molina and K. Salem. Sagas. In *ACM SIGMOD*, 1987.
- [7] R. E. Strom and S. Yemini. Typestate: a programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, 12(1), 1986.
- [8] R. DeLine and M. Fähndrich. Typestates for objects. In *ECOOP*, 2004.
- [9] S. Gössner, G. Normington, and C. Bormann (eds.). *RFC 9535: JSONPath — Query Expressions for JSON*. IETF, 2024.
- [10] Cloud Native Computing Foundation. *Serverless Workflow Specification*. <https://serverlessworkflow.io/>.
- [11] Amazon Web Services. *AWS Step Functions Developer Guide / Amazon States Language specification*. <https://states-language.net/spec.html>.